

Database Architecture

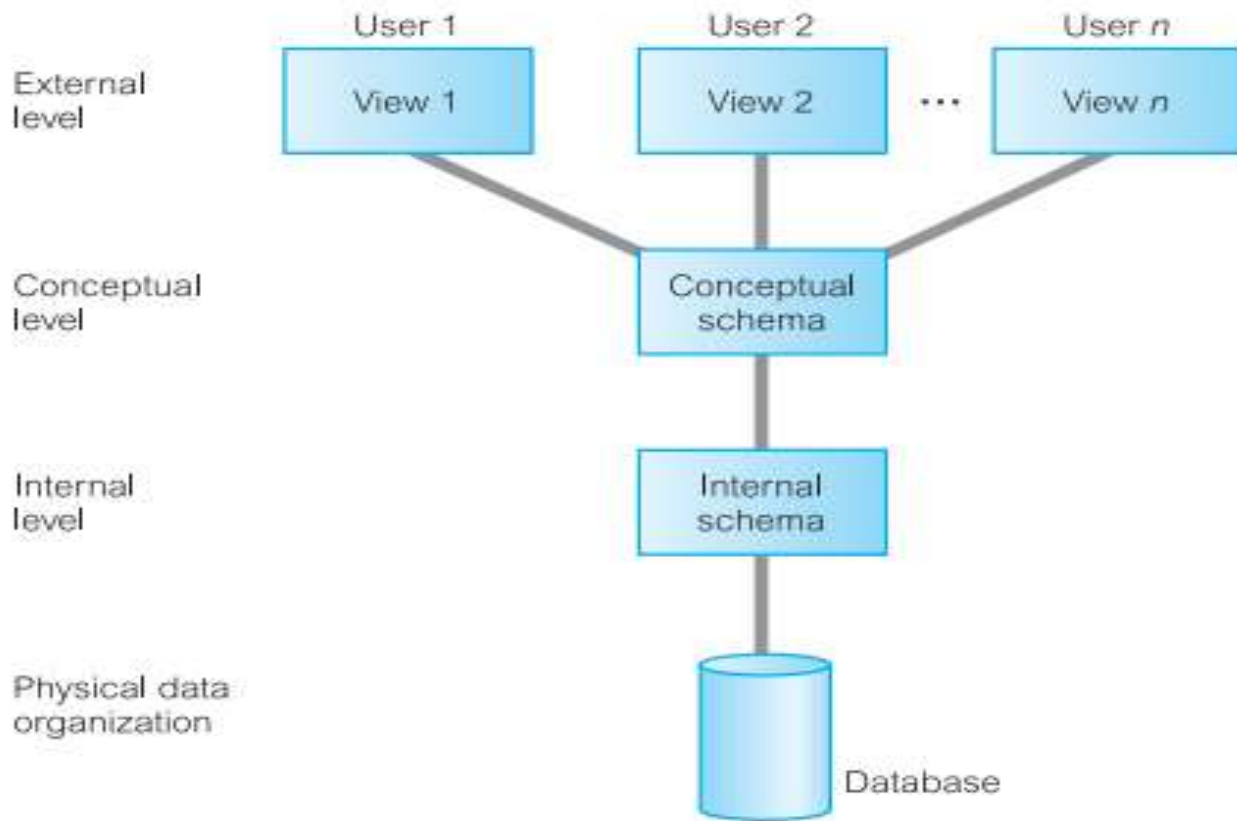
Chapter # 02

The Three-Level ANSI-SPARC Architecture

The distinction between the logical and physical representation of data were recognized in 1978 when ANSI/SPARC committee proposed a generalized framework for database systems.

This framework provided a three level architecture, three levels of abstraction at which the database could be viewed.

The Three-Level ANSI-SPARC Architecture



Need for Abstraction

The objective of the three-level architecture is to separate each user's view of the database from the way the database is physically represented. There are several reasons why this separation is desirable:

- Each user should be able to access the same data, but have a different customized view of the data. Each user should be able to change the way he or she views the data, and this change should not affect other users.
- Users should not have to deal directly with physical database storage details. In other words, a user's interaction with the database should be independent of storage considerations.
- The Database Administrator (DBA) should be able to change the database storage structures without affecting the users' views.

Need for Abstraction

- The internal structure of the database should be unaffected by changes to the physical aspects of storage, such as the changeover to a new storage device.
- The DBA should be able to change the conceptual structure of the database without affecting all users.

External Level

The users' view of the database. This level describes that part of the database that is relevant to each user

- The external level consists of a number of different external views of the database. Each user has a view of the 'real world' represented in a form that is familiar for that user.
- The external view includes only those entities, attributes, and relationships in the 'real world' that the user is interested in
- Different views may have different representations of the same data.
- Some views might include derived or calculated data: data not actually stored in the database as such, but created when needed

Conceptual Level

The community view of the database. This level describes what data level is stored in the database and the relationships among the data.

The middle level in the three-level architecture is the conceptual level. This level contains the logical structure of the entire database as seen by the DBA. It is a complete view of the data requirements of the organization that is independent of any storage considerations.

The conceptual level represents:

- **all entities, their attributes, and their relationships;**
- **the constraints on the data;**
- **semantic information about the data;**
- **security and integrity information.**

Conceptual Level

- The conceptual level supports each external view, in that any data available to a user must be contained in, or derivable from, the conceptual level.
- This level must not contain any storage-dependent details

External and conceptual layers



Internal Level

The physical representation of the database on the computer. This level describes how the data is stored in the database.

- The internal level covers the physical implementation of the database to achieve optimal runtime performance and storage space utilization
- It covers the data structures and file organizations used to store data on storage devices.
- It interfaces with the operating system access methods (file management techniques for storing and retrieving data records) to place the data on the storage devices, build the indexes, retrieve the data, and so on.

Internal Level

The internal level is concerned with such things as:

- storage space allocation for data and indexes
- record descriptions for storage (with stored sizes for data items);
- record placement;
- data compression and data encryption techniques.

The Three-Level Architecture

External view 1

sNo	fName	lName	age	salary
-----	-------	-------	-----	--------

External view 2

staffNo	lName	branchNo
---------	-------	----------

Conceptual level

staffNo	fName	lName	DOB	salary	branchNo
---------	-------	-------	-----	--------	----------

Internal level

```
struct STAFF {  
    int staffNo;  
    int branchNo;  
    char fName [15];  
    char lName [15];  
    struct date dateOf Birth;  
    float salary;  
    struct STAFF *next;  
};  
index staffNo; index branchNo;
```

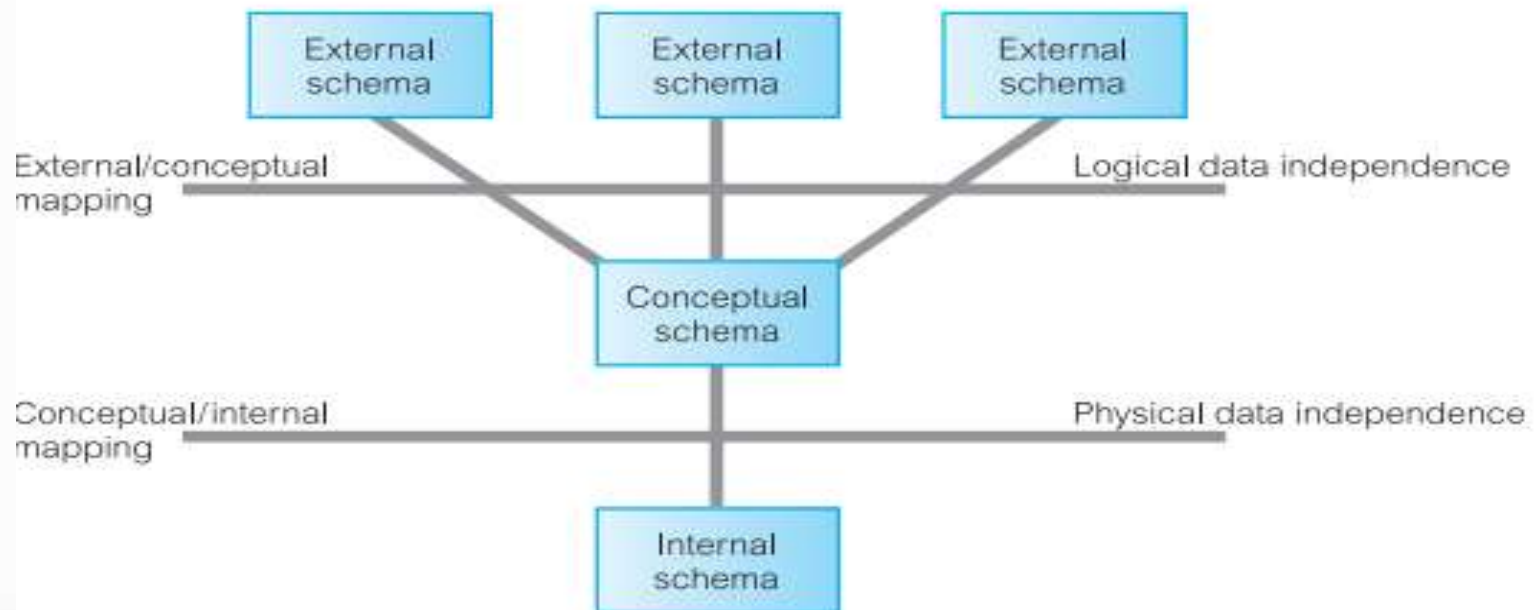
/* pointer to next Staff record */
/* define indexes for staff */

Data Independence

A major objective for the three-level architecture is to provide data independence, which means that upper levels are unaffected by changes to lower levels. There are two kinds of data independence:

logical data independence

physical data independence



Data Independence

Logical data Independence

Logical data independence refers to the immunity of the external independence schemas to changes in the conceptual schema

Changes to the conceptual schema, such as the addition or removal of new entities, attributes, or relationships, should be possible without having to change existing external schemas or having to rewrite application programs

The users for whom the changes have been made need to be aware of them, but what is important is that other users should not be.

Data Independence

Physical data independence

Physical data independence refers to the immunity of the conceptual independence schema to changes in the internal schema.

Changes to the internal schema, such as using different file organizations or storage structures, using different storage devices, modifying indexes, or hashing algorithms, should be possible without having to change the conceptual or external schemas

From the users' point of view, the only effect that may be noticed is a change in performance

Data Models

- A model is a representation of 'real world' objects and events, and their associations
- A data model represents the organization itself.
- It should provide the basic concepts and notations that will allow database designers and end-users unambiguously and accurately to communicate their understanding of the organizational data.

A data model can be thought of as comprising three components:

- (1) a **structural part**, consisting of a set of rules according to which databases can be constructed;

Data Models

- (2) **manipulative part**, defining the types of operation that are allowed on the data
(this includes the operations that are used for updating or retrieving data from the database and for changing the structure of the database)
- (3) possibly a set of integrity constraints, which ensures that the data is accurate.

Data models fall into three broad categories:

object-based, ***record-based***, and ***physical data models***.

The first two are used to describe data at the conceptual and external levels, the latter is used to describe data at the internal level.

Object Based Logical Model

- Object-based data models contains concepts such as entities, attributes, and relationships
- It was first developed by Peter Chen in 1976

Some of the more common types of object-based data model are:

Entity–Relationship

Functional

Object-Oriented.

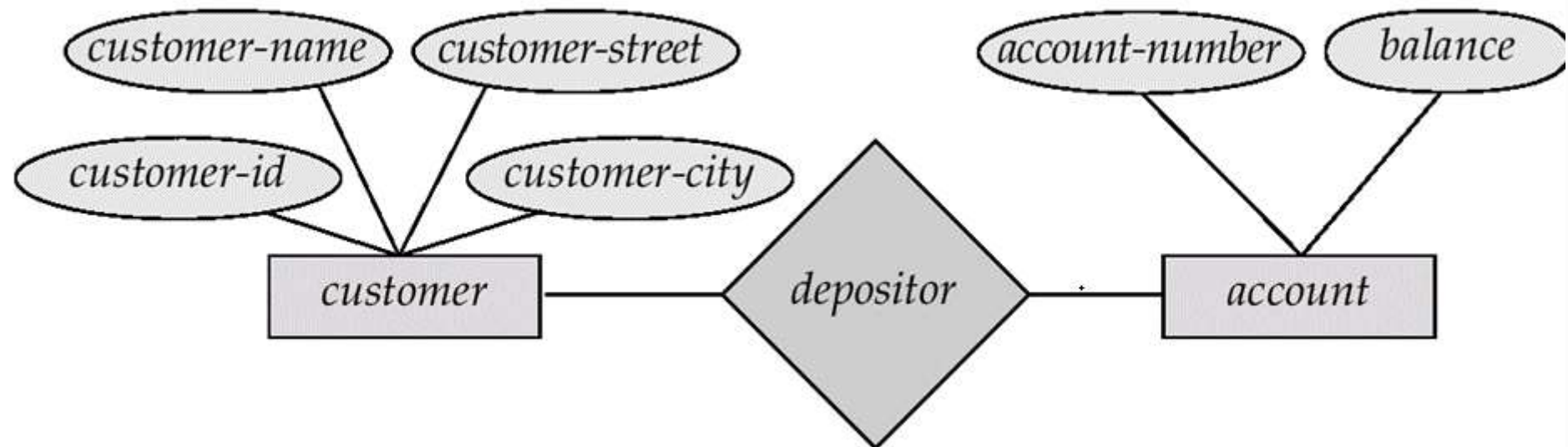
- An **entity** is a distinct object (a person, place, thing, concept, event) in the organization that is to be represented in the database.
- An **attribute** is a property that describes some aspect of the object that we wish to record.
- a **relationship** is an association between entities.

Object Based Logical Model

The object-oriented data model extends the definition of an entity to include not only the attributes that describe the state of the object but also the actions that are associated with the object, that is, its behavior. The object is said to encapsulate both state and behavior

Object based logical model is then transformed into record based logical model

Object Based Logical Model



Record Based Model

- Record Based Model or Logical Data Model or simply Database Model is logical conversion of Entity Relationship Model
- The database model visualizes the logical model ,which in turn is the basis for the physical implementation in the form of database tables indexes and other mechanism
In the database design
- In a record-based model, the database consists of a number of fixed-format records possibly of differing types.
- There are three principal types of record-based logical data model:
 - I. The relational data model*
 - II. The network data model*
 - III. The hierarchical data model.*
- Choice of Data model depends upon the relationship between various pieces of data

Record Based Model

Each model is best suited for particular class of problems depending on

- **Nature of the data relationships**
- **The need for flexibility**
- **The volume of changes to the database**

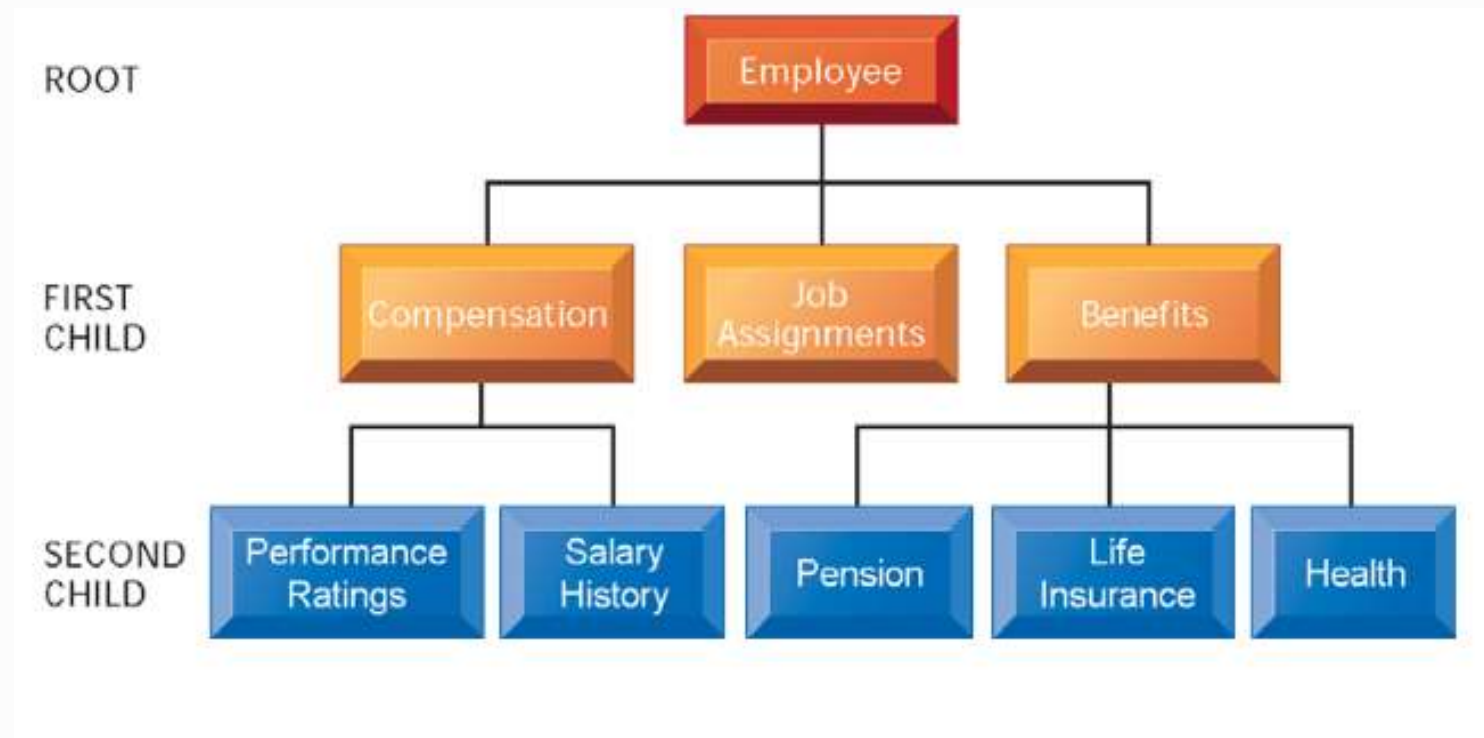
The hierarchical and network data models were developed almost a decade before the relational data model, so their links to traditional file processing concepts are more evident.

Record Based Model

Hierarchical Model

- The hierarchical DBMS is used to model one-to-many relationships, presenting data to users in a treelike structure
- Within each record, data elements are organized into pieces of records called segments.
- To the user, each record looks like an organizational chart with one top-level segment called the root
- An upper segment is connected logically to a lower segment in a parent–child relationship.
- A parent segment can have more than one child, but a child can have only one parent.
- hierarchical structures depict one-to-many relationships

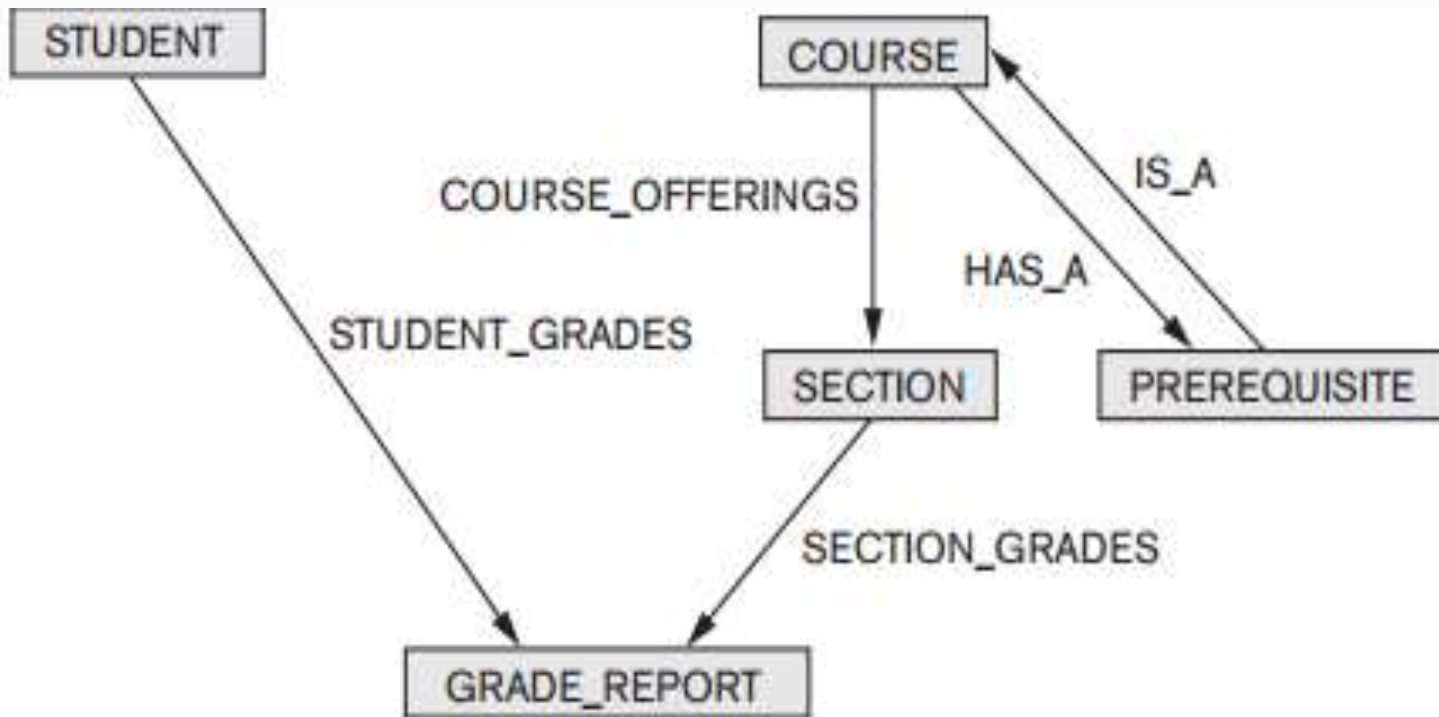
Record Based Model-*Hierarchical Model*



Record Based Model-*Network Model*

- In the network model, data is represented as collections of records, and relationships are represented by sets
- In network model parents can have multiple children, and a child can have more than one parent.
- Network DBMS depict data logically as many-to-many relationships
- A 1:N, or one-to-many, relationship relates one instance of a record to many record instances using some pointer linking mechanism in these model

Record Based Model-*Network Model*



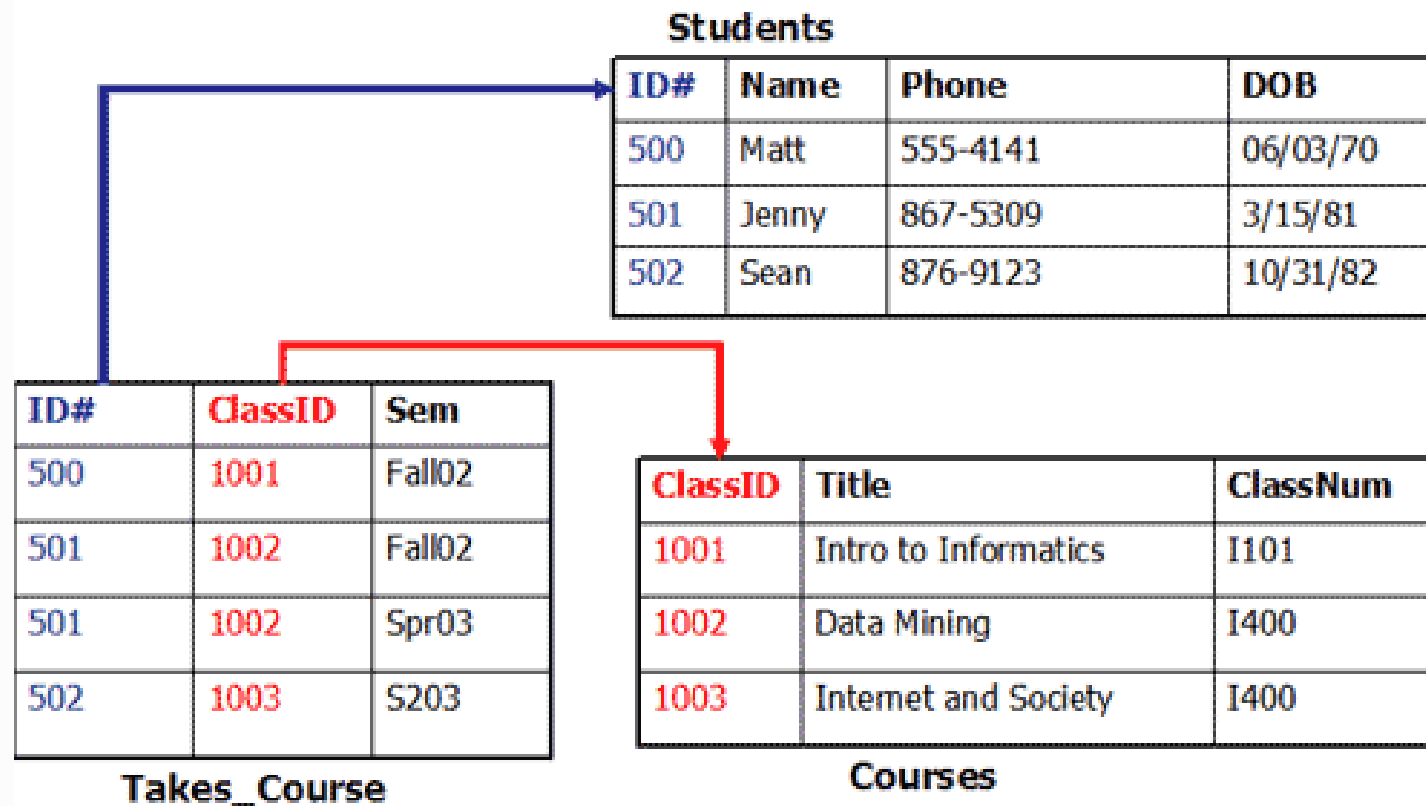
Relational Database Model

- The relational data model is based on the concept of mathematical relations
- In the relational model, data and relationships are represented as tables, each of which has a number of columns with a unique name
- Relational databases permits more flexible combination and easier retrieval of information than other models

Relation

A data in one table can be related to any piece of data in another table as long as both table share common element

Relational Database Model



Physical Database Model

- Physical data models describe how data is stored in the computer, representing information such as record structures, record orderings, and access path
- Its just **Data Definition Language(DDL)**
- DDL provides the semantics for administering all the physical objects
In your database that is stored in the data dictionary

The result of compilation of DDL Operation is a set of tables ,views ,roles, users and other objects as physical database